

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

PRÁCTICA EXPERIMENTAL – UNIDAD IV

Validación, Gestión de Requisitos y Herramientas CASE

ASIGNATURA:

INGENIERÍA DE REQUERIMIENTOS

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERÓN

INTEGRANTES:

CEDEÑO AVILA WINSTON DAMIAN, CI: 0942833492, wcedenoa2@uteq.edu.ec

CORDOVA CARREÑO MAYRA LUCILA, CI: 0750286775, mcordovac2@uteq.edu.ec

MENDOZA PÁRRAGA ANDY JOHEL, CI: 1251401590, amendozap9@uteq.edu.ec, Titular PFC

EQUIPO:

B

CURSO/PARALELO:

CUARTO SEMESTRE PARALELO “B”

SISTEMA PFC:

MundiPets (Andy Mendoza – Titular PFC)

REPOSITORIO GITHUB:

<https://github.com/andymendozap03/ISR401-PFC-ERS-EQUIPOB.git>

QUEVEDO – LOS RÍOS – ECUADOR
2026 – 2027 PPA

Índice

1.	Resumen y objetivos	3
1.1.	Objetivo general	3
1.2.	Objetivos específicos	3
2.	Fundamento teórico	4
2.1.	Marco normativo	4
2.1.1.	¿Qué regula la validación y gestión de requisitos?	4
2.1.2.	IEEE/ISO/IEC 29148:2018 – Proceso de validación de requisitos de software	4
2.1.3.	IEEE/ISO/IEC 15288:2023 y 12207:2017 – Procesos de control de cambios	4
2.1.4.	ISO/IEC 25010:2023 – Modelo de calidad aplicado al ERS	4
2.1.5.	SWEBOK v4.0 y Pohl (2025) Parte V	4
2.1.6.	El CCB según PMBOK 7. ^a edición	4
2.2.	Validación de requisitos	4
2.2.1.	Verificación vs. validación: diferencias conceptuales	4
2.2.2.	Principios de validación de requisitos	4
2.2.3.	Técnicas de validación	4
2.2.4.	Inspección Fagan: origen, fases y roles	4
2.2.5.	Métricas de inspección	4
2.2.6.	Caso de estudio o ejemplo aplicado	4
2.3.	Gestión de requisitos	4
2.3.1.	Gestión de la configuración del ERS	4
2.3.2.	Línea base (baseline) y control de versiones	4
2.3.3.	Atributos de los requisitos	4
2.3.4.	Trazabilidad: pre-traceability y post-traceability	4
2.3.5.	Matriz de trazabilidad	4
2.3.6.	Proceso de cambio: RFC, análisis de impacto, CCB, implementación y cierre	4
2.3.7.	Métricas de volatilidad de requisitos	4
2.4.	Herramientas CASE para IR	4
2.4.1.	Clasificación de herramientas CASE	4
2.4.2.	Funcionalidades esperables en gestión de requisitos	4
2.4.3.	Criterios de selección de una herramienta CASE	5
2.4.4.	Limitaciones y costos	5
2.5.	IR en procesos ágiles	5
2.5.1.	Product backlog, historias de usuario y criterios de aceptación	5
2.5.2.	Grooming y Definition of Ready / Definition of Done	5
2.5.3.	BDD y formato Gherkin	6
2.5.4.	Trazabilidad en entornos ágiles	6
2.5.5.	IR documental frente a IR ágil: ¿cuándo conviene cada enfoque?	6
3.	Inspección formal del ERS (método Fagan)	7
3.1.	Planificación y roles	7
3.2.	Preparación individual	7
3.3.	Acta de la reunión de inspección	7
3.4.	Métricas de inspección	7
4.	Correcciones aplicadas	7
5.	Gestión del cambio y Change Control Board (CCB)	7
5.1.	RFC-01 – Cambio de alcance (nuevo requisito)	7
5.2.	RFC-02 – Cambio de calidad (modificación de un RNF)	7
5.3.	RFC-03 – Cambio de restricción o normativa	7
5.4.	Acta del CCB	7
5.5.	Versión resultante del ERS	7
6.	Trazabilidad en herramienta CASE	7
6.1.	Estructura del tablero	7
6.2.	Campos personalizados	7

6.3.	Matriz de trazabilidad	7
6.4.	Evidencias del tablero	7
7.	Línea base del ERS con Git	7
7.1.	Repositorio y estructura	7
7.2.	Commits semánticos	7
7.3.	Tag anotado de línea base	7
7.4.	CHANGELOG.md	7
7.5.	Historial de commits (evidencia)	7
8.	Comparativa de herramientas CASE	7
9.	IR tradicional frente a IR ágil	7
10.	Conclusiones críticas	7
11.	Distribución del trabajo	7
12.	Referencias	7
Anexo A	— Lista de verificación de inspección (checklists individuales)	8
Anexo B	— Registro de defectos	9
Anexo C	— Formularios RFC y acta del CCB	10
Anexo D	— Exportación CSV del backlog	11
Anexo E	— Capturas del tablero CASE y del repositorio Git	12
Anexo F	— Referencias IEEE y declaración de uso de Inteligencia Artificial	13

1. Resumen y objetivos

1.1. Objetivo general

Aplicar técnicas formales de validación de requisitos, gestionar cambios mediante un Change Control Board (CCB) simulado, implementar la trazabilidad del ERS en una herramienta CASE, establecer una línea base con Git y comparar metodológicamente la Ingeniería de Requisitos (IR) tradicional frente a la IR ágil, sobre el ERS real del Proyecto Fin de Curso MundiPets.

1.2. Objetivos específicos

1. Ejecutar una inspección formal tipo Fagan sobre el ERS del PFC, con roles diferenciados, lista de verificación y registro de defectos.
2. Calcular e interpretar métricas de inspección (densidad de defectos, distribución por tipo y severidad, tasa de corrección).
3. Tramitar tres solicitudes formales de cambio (RFC) con análisis de impacto sustentado en la matriz de trazabilidad.
4. Deliberar y decidir en un CCB simulado, dejando acta motivada y versión actualizada del ERS.
5. Construir la trazabilidad bidireccional del ERS en una herramienta CASE de gestión (Jira o Trello) con campos personalizados y enlaces verificables.
6. Establecer y publicar la línea base del ERS con control de versiones (commit semántico, tag anotado y `CHANGELOG.md`).
7. Comparar herramientas CASE de IR y contrastar la IR tradicional frente a la IR ágil, cerrando con conclusiones críticas propias.

2. Fundamento teórico

2.1. Marco normativo

- 2.1.1. ¿Qué regula la validación y gestión de requisitos?
- 2.1.2. IEEE/ISO/IEC 29148:2018 – Proceso de validación de requisitos de software
- 2.1.3. IEEE/ISO/IEC 15288:2023 y 12207:2017 – Procesos de control de cambios
- 2.1.4. ISO/IEC 25010:2023 – Modelo de calidad aplicado al ERS
- 2.1.5. SWEBOK v4.0 y Pohl (2025) Parte V
- 2.1.6. El CCB según PMBOK 7.^a edición

2.2. Validación de requisitos

- 2.2.1. Verificación vs. validación: diferencias conceptuales
- 2.2.2. Principios de validación de requisitos
- 2.2.3. Técnicas de validación
- 2.2.4. Inspección Fagan: origen, fases y roles
- 2.2.5. Métricas de inspección
- 2.2.6. Caso de estudio o ejemplo aplicado

2.3. Gestión de requisitos

- 2.3.1. Gestión de la configuración del ERS
- 2.3.2. Línea base (baseline) y control de versiones
- 2.3.3. Atributos de los requisitos
- 2.3.4. Trazabilidad: pre-traceability y post-traceability
- 2.3.5. Matriz de trazabilidad
- 2.3.6. Proceso de cambio: RFC, análisis de impacto, CCB, implementación y cierre
- 2.3.7. Métricas de volatilidad de requisitos

2.4. Herramientas CASE para IR

Las herramientas CASE (*Computer-Aided Software Engineering*) constituyen el soporte tecnológico que automatiza, total o parcialmente, las actividades de elicitación, especificación, verificación, validación y gestión de los requisitos a lo largo del ciclo de vida del software. Su adopción responde a la necesidad de reducir el esfuerzo manual asociado a la trazabilidad, el control de versiones y la comunicación entre los interesados, actividades que, gestionadas sin apoyo automatizado, incrementan la probabilidad de inconsistencias en el ERS [1].

2.4.1. Clasificación de herramientas CASE

Las herramientas CASE aplicadas a la ingeniería de requisitos (IR) se clasifican habitualmente según la etapa del proceso que automatizan y el alcance funcional que ofrecen. Un análisis reciente de 56 herramientas de IR agrupó sus características en siete perspectivas complementarias: gestión de proyectos, especificación, colaboración, personalización, interoperabilidad, metodología y soporte al usuario, evidenciando que ninguna herramienta cubre de manera uniforme la totalidad de estas dimensiones [2]. Desde una perspectiva funcional más orientada al mercado, otros estudios distinguen entre herramientas de *requirements gathering* (apoyadas en técnicas visuales, diagramas de contexto, descomposición funcional e historias de usuario) y herramientas de *requirements management* propiamente dichas, orientadas al control de cambios, la trazabilidad y la generación de reportes [3]. Esta doble clasificación por etapa del proceso y por alcance funcional resulta coherente con el proceso de validación y gestión descrito en el estándar ISO/IEC/IEEE 29148:2018 y con los procesos de control de cambios de la ISO/IEC/IEEE 12207:2017 [1, 4].

2.4.2. Funcionalidades esperables en gestión de requisitos

Con independencia de la clasificación adoptada, la literatura converge en un conjunto de funcionalidades mínimas esperables en una herramienta CASE orientada a la gestión de requisitos. Ozkaya et al. [2] identifican, entre las más recurrentes, la especificación estructurada de requisitos, el versionado, la trazabilidad bidireccional, el soporte a la colaboración distribuida entre analistas y stakeholders, y la interoperabilidad con otras herramientas del ciclo de vida (gestión de pruebas, control de código y gestión de proyectos). En la misma línea, la

comparación cualitativa de diez herramientas comerciales de gestión de requisitos realizada por Nadeem et al. [3] evalúa explícitamente diez parámetros –seguimiento de errores, gestión jerárquica de requisitos, diseño visual, gestión de riesgos, gestión de pruebas, verificación de requisitos, análisis de calidad, versionado, gestión de *releases* y generación de reportes–, concluyendo que ninguna herramienta analizada obtiene un desempeño uniformemente alto en todos ellos, lo que refuerza la necesidad de un criterio de selección alineado con las prioridades específicas del proyecto.

2.4.3. Criterios de selección de una herramienta CASE

La selección de una herramienta CASE no es un problema trivial: involucra criterios técnicos, organizacionales y económicos que deben ponderarse según el contexto del proyecto. Bjarnason et al. [5] proponen un modelo de selección de software a gran escala construido a partir de revisiones rápidas e iterativas, que combina una lista de criterios de evaluación (funcionales, de usabilidad, de soporte del proveedor y de costo total de propiedad) con un proceso estructurado para identificar, valorar y filtrar alternativas antes de la adopción definitiva. Este enfoque es aplicable directamente a la selección de herramientas de gestión de requisitos, dado que ambos dominios comparten la dificultad de comparar productos con conjuntos de características parcialmente solapados y una oferta comercial heterogénea. Entre los criterios más citados en la literatura destacan la facilidad de integración con el resto del ecosistema de herramientas del proyecto, la curva de aprendizaje, el soporte a estándares de la industria y la escalabilidad para equipos distribuidos [2, 5].

2.4.4. Limitaciones y costos

La adopción de herramientas CASE no está exenta de limitaciones. En primer lugar, el costo total de propiedad –licenciamiento, capacitación del equipo y mantenimiento– constituye una barrera relevante, particularmente para organizaciones pequeñas o proyectos con presupuestos ajustados [5]. En segundo lugar, el análisis comparativo de Nadeem et al. [3] evidencia que las herramientas orientadas a funcionalidades avanzadas (gestión de riesgos, análisis de calidad de requisitos) suelen sacrificar simplicidad de uso, lo que puede traducirse en una curva de adopción prolongada. Finalmente, Ozkaya et al. [2] señalan que la interoperabilidad entre herramientas de distintos proveedores continúa siendo una limitación estructural del mercado, lo que obliga frecuentemente a las organizaciones a asumir integraciones a medida o a tolerar silos de información entre la especificación de requisitos y otras disciplinas del ciclo de vida (pruebas, arquitectura, gestión de proyectos).

2.5. IR en procesos ágiles

La incorporación de prácticas ágiles ha transformado sustancialmente la forma en que se elicitán, documentan y gestionan los requisitos, desplazando el énfasis desde la especificación exhaustiva y previa hacia una construcción incremental y colaborativa del entendimiento del producto. Esta sección revisa los artefactos, prácticas y mecanismos de trazabilidad propios de la IR ágil, y los contrasta con el enfoque documental descrito en 3.1–3.3.

2.5.1. Product backlog, historias de usuario y criterios de aceptación

El *product backlog* constituye el artefacto central de la gestión ágil de requisitos: una lista ordenada y viva de necesidades del producto que se refina de manera continua. Su unidad de trabajo predominante es la historia de usuario, cuyo origen se remonta a la programación extrema y que se popularizó como mecanismo para capturar requisitos desde la perspectiva del usuario final, priorizando la conversación sobre la documentación exhaustiva [6]. No obstante, la sola redacción de una historia de usuario no garantiza su calidad: el estudio de caso de Kuhail y Lauesen [7], que evaluó las respuestas de ocho profesionales de TI frente a un proyecto real utilizando los criterios de calidad de la IEEE 830 (completitud, corrección, verificabilidad y trazabilidad), encontró que las historias de usuario producidas cubrieron apenas el 33 % de las necesidades identificadas en el informe de análisis original, lo que evidencia el riesgo de pérdida de información al migrar de una especificación documental a un formato ágil sin mecanismos de control adicionales. Por ello, Ferreira et al. [6] proponen enriquecer las historias de usuario con criterios de aceptación explícitos, épicas y temas que permitan preservar la trazabilidad conceptual entre el requisito de alto nivel y su implementación incremental, en línea con los atributos de calidad definidos en la ISO/IEC/IEEE 29148:2018 [1].

2.5.2. Grooming y Definition of Ready / Definition of Done

El refinamiento del backlog (*backlog grooming* o *refinement*) es la actividad continua mediante la cual el equipo desglosa, estima y clarifica los elementos del backlog hasta que cumplen un nivel de detalle suficiente para ser abordados en un sprint. La Guía de Scrum no define formalmente una *Definition of Ready* (DoR), pero establece que los elementos del backlog deben alcanzar un grado de transparencia y comprensión que permita completarlos dentro de un sprint [8]. La *Definition of Done* (DoD), en cambio, sí forma parte del marco oficial de Scrum y constituye un compromiso formal y compartido por el equipo sobre lo que significa que un incremento esté

terminado, siendo responsabilidad primaria del equipo de desarrollo mantener su cumplimiento [8]. Tomaselli et al. [9], en su revisión sistemática sobre métodos de gestión de requisitos en desarrollo ágil, destacan que la ausencia de criterios de entrada y salida explícitos –es decir, de un DoR y un DoD bien definidos– es una de las causas recurrentes de retrabajo y de defectos de calidad en los incrementos entregados, y recomiendan su integración temprana como práctica de gestión de requisitos dentro del marco de calidad ágil (*Agile Quality Framework*).

2.5.3. BDD y formato Gherkin

El desarrollo guiado por comportamiento (*Behavior-Driven Development*, BDD) extiende la lógica de las historias de usuario hacia especificaciones ejecutables, escritas en lenguaje natural estructurado mediante el formato Gherkin (*Given-When-Then*), que permite a analistas, desarrolladores y evaluadores compartir un mismo artefacto como documentación, criterio de aceptación y base para pruebas automatizadas. La revisión sistemática de Abushama et al. [10] sobre el efecto del desarrollo guiado por pruebas (TDD) y del BDD en los factores de éxito de un proyecto concluye que ambas prácticas inciden positivamente en la satisfacción del cliente y en la reducción de defectos, atribuyendo al BDD un valor adicional en la mejora de la comunicación entre perfiles técnicos y no técnicos gracias a la legibilidad de sus escenarios. Esta característica convierte al formato Gherkin en un mecanismo natural de validación de requisitos en entornos ágiles, ya que cada escenario actúa simultáneamente como criterio de aceptación verificable y como evidencia de trazabilidad entre la necesidad del negocio y el comportamiento implementado del sistema.

2.5.4. Trazabilidad en entornos ágiles

La trazabilidad de requisitos –la capacidad de seguir un requisito desde su origen hasta su implementación y verificación– presenta desafíos particulares en contextos ágiles, donde la reducción deliberada de la documentación formal puede debilitar los enlaces explícitos entre artefactos. El estudio de mapeo de Charalampidou et al. [11] sobre trazabilidad de software identifica que, si bien la mayoría de los métodos de trazabilidad fueron concebidos originalmente para procesos secuenciales y documentales, existe una tendencia creciente hacia enfoques ligeros y semiautomatizados basados en recuperación de información y en la vinculación de commits, historias de usuario y casos de prueba que se adaptan mejor a la cadencia iterativa del desarrollo ágil. En la misma dirección, Loh Mun Keong y Che Embi [12] advierten que la documentación de requisitos no funcionales es una de las prácticas más frecuentemente descuidadas en equipos ágiles, lo que compromete la trazabilidad de atributos de calidad como los definidos en la ISO/IEC 25010:2023 [13] y exige mecanismos complementarios (etiquetado de historias, matrices ligeras o herramientas CASE integradas) para no perder dicha trazabilidad frente a la mayor velocidad de entrega.

2.5.5. IR documental frente a IR ágil: ¿cuándo conviene cada enfoque?

La disyuntiva entre un enfoque documental –propio de los estándares revisados en 3.1 a 3.3, con énfasis en el ERS, la línea base y la matriz de trazabilidad formal– y un enfoque ágil –centrado en el backlog vivo, las historias de usuario y la trazabilidad ligera– no debe entenderse como una elección excluyente, sino como una decisión de diseño de proceso condicionada por el contexto del proyecto. La revisión de Tomaselli et al. [9] concluye que los proyectos con alta variabilidad de requisitos, plazos cortos y participación constante del cliente se benefician de prácticas ágiles de gestión de requisitos, mientras que proyectos regulados, de misión crítica o con requisitos contractuales estrictos –por ejemplo, aquellos sujetos a normas como la ISO/IEC/IEEE 29148:2018 o a certificaciones sectoriales– requieren preservar un nivel de documentación formal y trazabilidad verificable que los formatos puramente ágiles no garantizan por defecto [1]. En la práctica, numerosas organizaciones adoptan esquemas híbridos: mantienen una especificación de alto nivel documentada y una línea base contractual, mientras gestionan el detalle operativo mediante backlog, historias de usuario y escenarios Gherkin, combinando así la trazabilidad formal exigida por el marco normativo con la agilidad necesaria para responder a los cambios propios del desarrollo iterativo [9, 12].

3. Inspección formal del ERS (método Fagan)

3.1. Planificación y roles

3.2. Preparación individual

3.3. Acta de la reunión de inspección

3.4. Métricas de inspección

4. Correcciones aplicadas

5. Gestión del cambio y Change Control Board (CCB)

5.1. RFC-01 – Cambio de alcance (nuevo requisito)

5.2. RFC-02 – Cambio de calidad (modificación de un RNF)

5.3. RFC-03 – Cambio de restricción o normativa

5.4. Acta del CCB

5.5. Versión resultante del ERS

6. Trazabilidad en herramienta CASE

6.1. Estructura del tablero

6.2. Campos personalizados

6.3. Matriz de trazabilidad

6.4. Evidencias del tablero

7. Línea base del ERS con Git

7.1. Repositorio y estructura

7.2. Commits semánticos

7.3. Tag anotado de línea base

7.4. CHANGELOG.md

7.5. Historial de commits (evidencia)

8. Comparativa de herramientas CASE

9. IR tradicional frente a IR ágil

10. Conclusiones críticas

11. Distribución del trabajo

12. Referencias

Anexo A — Lista de verificación de inspección (checklists individuales)

Anexo B — Registro de defectos

Anexo C — Formularios RFC y acta del CCB

Anexo D — Exportación CSV del backlog

Anexo E — Capturas del tablero CASE y del repositorio Git

Anexo F — Referencias IEEE y declaración de uso de Inteligencia Artificial

Referencias

- [1] ISO/IEC/IEEE, “ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering – Life Cycle Processes – Requirements Engineering,” International Organization for Standardization, inf. téc., 2018.
- [2] M. Ozkaya, G. Kardas y M. A. Kose, “An Analysis of the Features of Requirements Engineering Tools,” *Systems*, vol. 11, n.º 12, pág. 576, 2023. DOI: 10.3390/systems11120576 dirección: <https://doi.org/10.3390/systems11120576>
- [3] M. A. Nadeem, S. U.-J. Lee y M. U. Younus, “A Comparison of Recent Requirements Gathering and Management Tools in Requirements Engineering for IoT-Enabled Sustainable Cities,” *Sustainability*, vol. 14, n.º 4, pág. 2427, 2022. DOI: 10.3390/su14042427 dirección: <https://doi.org/10.3390/su14042427>
- [4] ISO/IEC/IEEE, “ISO/IEC/IEEE 12207:2017 – Systems and Software Engineering – Software Life Cycle Processes,” International Organization for Standardization, inf. téc., 2017.
- [5] E. Bjarnason, K. Sharma y P. Runeson, “Software Selection in Large-Scale Software Engineering: A Model and Criteria Based on Interactive Rapid Reviews,” *Empirical Software Engineering*, vol. 28, n.º 2, pág. 51, 2023. DOI: 10.1007/s10664-023-10288-w dirección: <https://doi.org/10.1007/s10664-023-10288-w>
- [6] A. Ferreira, A. R. Da Silva y A. C. R. Paiva, “Towards the Art of Writing Agile Requirements with User Stories, Acceptance Criteria, and Related Constructs,” en *Proceedings of the 17th International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE 2022)*, 2022, págs. 477-484. DOI: 10.5220/0011082000003176 dirección: <https://doi.org/10.5220/0011082000003176>
- [7] M. A. Kuhail y S. Lauesen, “User Story Quality in Practice: A Case Study,” *Software*, vol. 1, n.º 3, págs. 223-243, 2022. DOI: 10.3390/software1030010 dirección: <https://doi.org/10.3390/software1030010>
- [8] K. Schwaber y J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*, 2020. dirección: <https://scrumguides.org/>
- [9] G. P. Tomaselli, N. S. Pinto y C. Acuña, “Requirements Management Methods and Practices for Improving the Quality of Agile Software Development Processes: A Review of the Literature,” *Requirements Engineering*, vol. 30, págs. 371-398, 2025. DOI: 10.1007/s00766-025-00448-3 dirección: <https://doi.org/10.1007/s00766-025-00448-3>
- [10] H. M. Abushama, H. A. Alassam y F. A. Elhaj, “The Effect of Test-Driven Development and Behavior-Driven Development on Project Success Factors: A Systematic Literature Review Based Study,” en *2020 International Conference on Computer, Control, Electrical, and Electronics Engineering (ICCCEEE)*, 2021, págs. 1-9. DOI: 10.1109/ICCCEEE49695.2021.9429593 dirección: <https://doi.org/10.1109/ICCCEEE49695.2021.9429593>
- [11] S. Charalampidou, A. Ampatzoglou, E. Karountzos y P. Avgeriou, “Empirical Studies on Software Traceability: A Mapping Study,” *Journal of Software: Evolution and Process*, vol. 33, n.º 2, e2294, 2021. DOI: 10.1002/smr.2294 dirección: <https://doi.org/10.1002/smr.2294>
- [12] S. Loh Mun Keong y Z. Che Embi, “A Systematic Review on Non-Functional Requirements Documentation in Agile Methodology,” *Journal of Informatics and Web Engineering*, vol. 1, n.º 2, págs. 19-29, 2022. DOI: 10.33093/jiwe.2022.1.2.2 dirección: <https://doi.org/10.33093/jiwe.2022.1.2.2>
- [13] ISO/IEC, “ISO/IEC 25010:2023 – Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – Product Quality Model,” International Organization for Standardization, inf. téc., 2023.

Uso de la IA